# Parcial 1 - Lenguajes de Programación

Julián David Cristancho Bustos
Ciencias de la Computación e Inteligencia Artificial
Universidad Sergio Arboleda

10 de Marzo de 2026

## Punto 1: AFD para movimientos de ajedrez

**Expresión Regular:**
Analizando los ejemplos (`p->k4` y `kbp X qn`), la expresión regular definida es:

$$\hat{}[a-z]+(->\mid \text{X })[a-z]+[0-9]*\$$$

**Explicación de la solución:** Para la implementación en Python, se construyó un Autómata Finito Determinista (AFD) estricto. En lugar de utilizar expresiones regulares nativas o múltiples condicionales (`if/else`), se optó por un enfoque formal utilizando una matriz de transiciones.

1. Se mapeó el alfabeto de entrada a columnas de una matriz (letras, números, guion, mayor que, equis mayúscula y espacio).
2. Se definieron 10 estados (filas de la matriz), donde el estado 0 es el inicial, los estados 7 y 8 son de aceptación, y el 9 es un estado sumidero (error).
3. El programa itera sobre cada carácter de la cadena, actualizando el estado actual consultando directamente la matriz. Si al finalizar el recorrido el estado es 7 u 8, la cadena es aceptada.

**Paso a paso de ejecución:**

1. Guardar el código fuente en un archivo llamado `punto1.py`.
2. Abrir la terminal de Linux y navegar hasta la carpeta contenedora.
3. Ejecutar el comando: `python3 punto1.py`
4. Observar la salida en consola verificando el estado de aceptación de las pruebas.

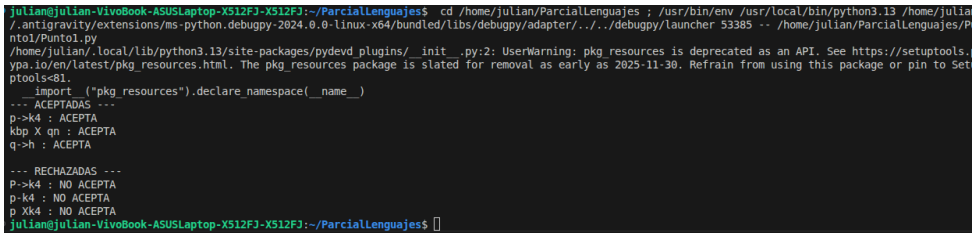


Figura 1: Pruebas de aceptación y rechazo del AFD para ajedrez en consola.

# Punto 2: AFD para Identificadores (IDs)

**Expresión Regular:**

$$[A - Za - z][A - Za - z0 - 9]*$$

**Explicación de la solución:** Al igual que en el punto anterior, se implementó el AFD en Python mediante una tabla de transiciones para optimizar el tiempo de ejecución y apegarse a la teoría de autómatas.

1. Se crearon tres estados: `0` (inicial), `1` (aceptación) y `2` (sumidero/error).
2. La matriz determina que si en el estado 0 llega una letra, pasa al estado 1. Si llega un número, cae al estado 2.
3. Estando en el estado 1, cualquier letra o número lo mantiene en el estado 1.
4. Se ejecutaron 5 pruebas (3 de aceptación y 2 de rechazo) demostrando el correcto funcionamiento del modelo sin utilizar lógica condicional en las transiciones.

**Paso a paso de ejecución:**

1. Guardar el código en el archivo `punto2.py`.
2. En la terminal de Linux, ejecutar el comando: `python3 punto2.py`
3. Verificar que se impriman en consola los resultados de las 5 pruebas solicitadas.

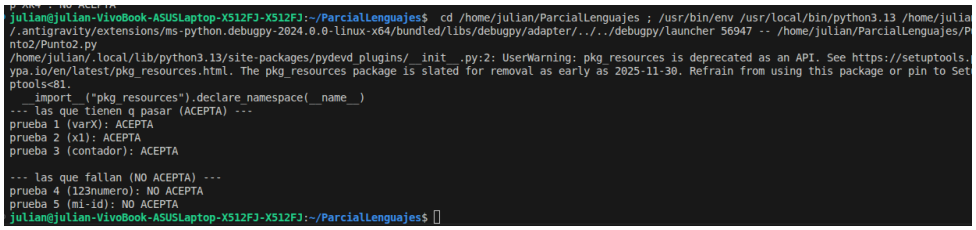


Figura 2: Ejecución de las 5 pruebas solicitadas para los IDs.

# Punto 3: Calculadora de Raíz Cuadrada con Flex y Bison

**Explicación de la solución:** Se desarrolló un analizador léxico en `Flex` y un analizador sintáctico en `Bison` (`C`) que leen operaciones matemáticas desde un archivo de texto.

1. **Lexer (`calc.l`):** Reconoce números reales, la palabra reservada `raiz` y los operadores aritméticos básicos.

2. **Parser (`calc.y`):** Define la gramática y respeta la precedencia de operadores (multiplicación y división antes que suma y resta).

3. **Método Numérico:** Para calcular la raíz cuadrada sin depender de la librería `math.h`, se implementó una función manual en C utilizando el método de Newton-Raphson. Se usó la fórmula iterativa $x_{n+1} = 0{,}5 \cdot (x_n + \frac{N}{x_n})$ con un ciclo iterativo hasta alcanzar una tolerancia de convergencia adecuada (0,000001).

4. El programa se compila uniendo el código generado por Flex y Bison con `gcc`, y la salida de las operaciones se muestra por consola leyendo línea a línea el `.txt`.

**Paso a paso de ejecución:**

1. Crear el archivo de texto de entrada con operaciones matemáticas, por ejemplo: `prueba.txt`.

2. Generar el código C del analizador léxico: `flex calc.l`

3. Generar el código C y cabeceras del analizador sintáctico: `bison -d calc.y`

4. Compilar los archivos generados con GCC: `gcc lex.yy.c calc.tab.c -o calculadora`

5. Ejecutar el programa pasándole el archivo de texto: `./calculadora prueba.txt`

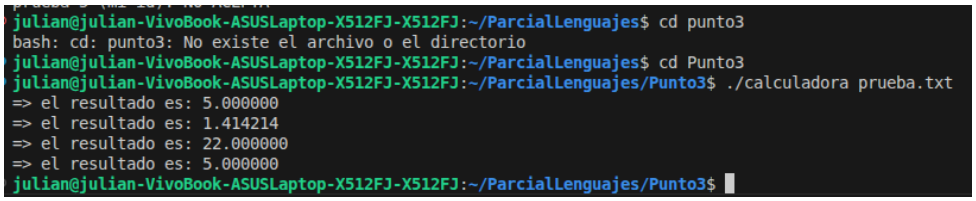


Figura 3: Resultados de la calculadora leyendo el archivo de texto.

# Punto 4: Rendimiento Algoritmo de Euclides (C vs Haskell)

**Explicación de la solución:** Se implementó el algoritmo recursivo para calcular el Máximo Común Divisor (MCD) en C (paradigma imperativo) y en Haskell (paradigma declarativo). Para poder medir el rendimiento real, se forzó la ejecución del algoritmo 50 millones de veces en ambos lenguajes.

**Análisis de resultados:**

- **C:** Al ser un lenguaje de bajo nivel compilado directamente a código máquina, la ejecución de los ciclos y las llamadas a memoria es notablemente más rápida.
- **Haskell:** Al ser un lenguaje con evaluación perezosa (*lazy evaluation*), fue necesario forzar la impresión del resultado final; de lo contrario, el compilador omitía el cálculo. Los tiempos de ejecución fueron más altos debido al *overhead* del lenguaje funcional.
- **Recursividad de cola (*Tail Recursion*):** En ambos lenguajes se evitó el desbordamiento de pila (*Stack Overflow*) gracias a que el algoritmo termina su evaluación en la misma llamada recursiva, lo cual es optimizado por los compiladores.

**Paso a paso de ejecución:**

1. Compilar el archivo en C: `gcc euclides.c -o euclides_c`
2. Ejecutar el binario de C para tomar la medición: `./euclides_c`
3. Compilar el código en Haskell usando el compilador GHC: `ghc euclides.hs -o euclides_hs`
4. Ejecutar el binario de Haskell para comparar: `./euclides_hs`

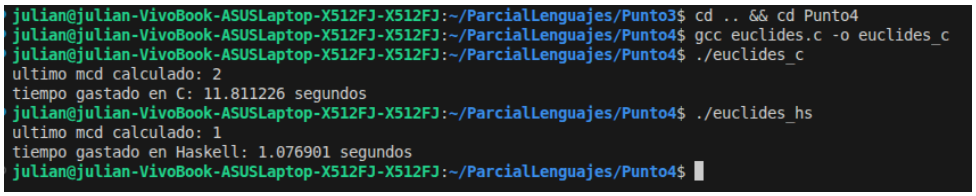


Figura 4: Comparación de tiempos de ejecución entre C y Haskell en consola.

# Punto 5: Serie de Maclaurin con ANTLR

**Explicación de la solución:** Se construyó un programa para calcular $e^x$ utilizando ANTLR4 y Python3.

1. **Gramática (`Maclaurin.g4`):** Se definió una gramática sencilla que reconoce la estructura `calcular(x, n)`, donde `x` es el exponente y `n` es la cantidad de términos de la serie.
2. **Generación y Visitor:** Se compilaron los archivos de ANTLR para Python3 y se sobrescribió la clase `Visitor`.
3. **Lógica matemática:** Dentro del Visitor de Python, se extraen los tokens numéricos y se aplica la sumatoria de la serie de Maclaurin: $\sum_{i=0}^{n-1} \frac{x^i}{i!}$.
4. **Resolución de entorno:** Durante el desarrollo se gestionaron adecuadamente las versiones de la librería de *runtime* de ANTLR en Python (forzando la compatibilidad a la versión 4.9.3) para asegurar la ejecución correcta del árbol (*ATN*) generado.

**Paso a paso de ejecución:**

1. Asegurar tener la versión correcta del *runtime* instalada:
   `pip3 install antlr4-python3-runtime==4.9.3 --break-system-packages`

2. Generar los archivos Lexer, Parser y Visitor desde la gramática:
   `antlr4 -Dlanguage=Python3 -visitor Maclaurin.g4`

3. Ejecutar el script principal que levanta el entorno interactivo:
   `python3 resolver.py`

4. Ingresar una prueba al momento de salir el prompt, por ejemplo: `calcular(2, 10)`.

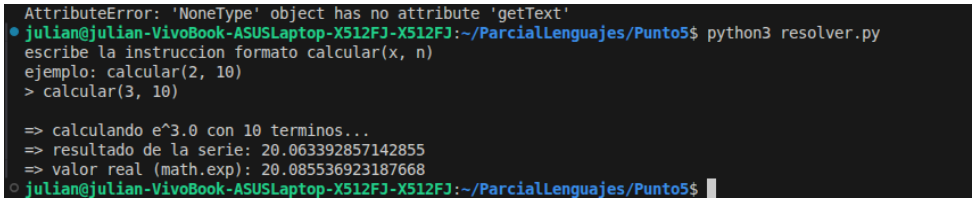


Figura 5: Ejecución del parser en Python resolviendo la serie de Maclaurin.